SESSION 10

UNION & UNION ALL

COMBINING APP ORDERS AND IN-STORE ORDERS

Goal: Solve all four tasks using MySQL-compatible SQL, grey code boxes, and simple explanations.

Core rule: UNION removes duplicate result rows. UNION ALL keeps all result rows, including duplicates.

Important: UNION and UNION ALL combine query results vertically. Each SELECT must return the same number of columns in the same order, with compatible data types.

Task 1 — Create the Tables and Insert Sample Records

Create AppOrders for food delivery app orders and InStoreOrders for direct restaurant orders. Each table contains order_id, customer_name, amount, and order_date. Three rows are inserted into each table. Ahad appears in both tables so the duplicate-name example is ready for Task 4.

```
CREATE DATABASE IF NOT EXISTS union_practice;
USE union_practice;

CREATE TABLE AppOrders (
    order_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    amount DECIMAL(10,2),
    order_date DATE
);

CREATE TABLE InStoreOrders (
    order_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    amount DECIMAL(10,2),
    order_date DATE
);

INSERT INTO AppOrders
    (order_id, customer_name, amount, order_date)
VALUES
    (1, 'Ahad', 250.00, '2026-10-20'),
    (2, 'Rahul', 350.00, '2026-10-21'),
    (3, 'Sana', 180.00, '2026-10-22');

INSERT INTO InStoreOrders
    (order_id, customer_name, amount, order_date)
VALUES
    (101, 'Ahad', 300.00, '2026-10-20'),
    (102, 'Priya', 220.00, '2026-10-21'),
    (103, 'Ayesha', 450.00, '2026-10-23');

SELECT * FROM AppOrders;
SELECT * FROM InStoreOrders;
```

Explanation

- `CREATE DATABASE IF NOT EXISTS` creates the practice database only if it does not already exist.
- `USE union_practice` selects the database where the tables will be created.
- `order_id INT PRIMARY KEY` gives each order a unique ID within its table.
- `customer_name VARCHAR(100)` stores names; `amount DECIMAL(10,2)` stores money with two decimal places; `order_date DATE` stores the date.
- `INSERT INTO ... VALUES` inserts three sample records into each table. The two tables use compatible column structures so they can be combined.

Task 2 — UNION: Get Unique Customer Names

Use UNION to combine names from both tables into one list. If a name appears in both tables, it appears only once in the output.

```
SELECT customer_name
FROM AppOrders

UNION

SELECT customer_name
FROM InStoreOrders
ORDER BY customer_name;
```

Expected output for the Task 1 sample data

customer_name
Ahad
Ayesha
Priya
Rahul
Sana

There are 6 name rows across the two tables, but only 5 unique names because Ahad appears in both. UNION removes the repeated result value. ORDER BY customer_name sorts the list alphabetically.

Task 3 — UNION ALL: Display Every Order

Use UNION ALL to combine all order records from the two tables. This query returns all four requested columns from both sources.

```
SELECT
    order_id,
    customer_name,
    amount,
    order_date
FROM AppOrders

UNION ALL

SELECT
    order_id,
    customer_name,
    amount,
    order_date
FROM InStoreOrders;
```

Explanation

The first SELECT returns every AppOrders row. UNION ALL appends the InStoreOrders rows without removing duplicates. Both SELECT statements return the same four columns in the same order. With the initial sample data, the result contains 6 rows: 3 app orders plus 3 in-store orders.

Note: The sample orders have different order_id values. Even if two orders have the same customer_name, they are still different order records.

Task 4 — Demonstrate the Difference

Insert one more order into each table using the same customer name, Karan. Then run two name-only queries to compare row counts. Selecting only customer_name is important here: UNION removes duplicate selected names, not records that merely share a name when other selected columns differ.

```
INSERT INTO AppOrders
    (order_id, customer_name, amount, order_date)
VALUES
    (4, 'Karan', 200.00, '2026-10-24');

INSERT INTO InStoreOrders
    (order_id, customer_name, amount, order_date)
VALUES
    (104, 'Karan', 250.00, '2026-10-24');
```

A. Run UNION

```
SELECT customer_name
FROM AppOrders
UNION
SELECT customer_name
FROM InStoreOrders;
```

Expected result count: 6 unique names — Ahad, Rahul, Sana, Priya, Ayesha, and Karan.

B. Run UNION ALL

```
SELECT customer_name
FROM AppOrders
UNION ALL
SELECT customer_name
FROM InStoreOrders;
```

Expected result count: 8 rows — four customer-name rows from each table. Ahad appears twice and Karan appears twice.

Result count comparison

UNION returns 6 rows: the 8 name rows reduce to 6 unique names after Karan is added. UNION ALL returns 8 rows because it keeps all four names from each table, including repeated Ahad and Karan.

Key detail: UNION compares the entire row returned by each SELECT. In the Task 3 query, order IDs, amounts, and dates are included, so orders with the same customer name but different details are not duplicates of the full row.

Final Revision

Feature	UNION	UNION ALL
Combines SELECT results	Yes	Yes
Removes duplicate result rows	Yes	No
Can return fewer rows due to duplicates	Yes	No
Column count/order requirement	Same compatible columns	Same compatible columns

Remember: Use UNION when you want distinct result rows. Use UNION ALL when you want to preserve every row from each query.